

Employee Travel Desk (ETD)

A Microservices-Based Corporate Travel Management Platform

Technical Project Documentation

Cognizant FSE Business-Aligned Project

Java 21 | Spring Boot 3.5 | Spring Cloud | MySQL 8 | Angular 17 | JWT

5 Spring Boot microservices • 5 Angular micro-frontends • MySQL

1. Project Overview

Employee Travel Desk (ETD) is a corporate travel-management platform that automates the complete business-trip lifecycle inside an organisation. It is built as a distributed system of five independent Spring Boot microservices on the backend and five matching Angular single-page micro-frontends on the front end, integrated through a shared, stateless JSON Web Token (JWT) and backed by MySQL.

The platform serves three user roles:

Role	Responsibilities
HR	Manages employees and grades, approves or rejects travel requests, and calculates the approved trip budget.
Employee	Raises travel requests, tracks their reservations, and submits post-trip reimbursement claims.
TravelDeskExe (Travel Desk Executive)	Uploads reservation bookings (flight / train / bus / cab / hotel) for approved trips and reviews / processes reimbursement claims.

End-to-end flow: an Employee raises a request; HR approves it and calculates the budget; the TravelDeskExe books the reservations; the Employee travels and submits invoices; and the TravelDeskExe approves or rejects each reimbursement claim.

2. System Architecture

Each backend microservice is autonomous, owns its data, and exposes a REST API documented with OpenAPI / Swagger. Each Angular app maps one-to-one to a backend service and runs on its own port. The browser talks to every service directly (open CORS); there is no central API gateway in the current build.

Microservice	Port	Front-end app (port)	Responsibility
account-management	8081	account-management-app (4201)	Employees, grades, grade history; owns the shared account_management MySQL database
auth-service	8080	auth-app (4200)	Central login, token refresh, logout, JWT blacklist check
travel-planner	8082	travel-planner-app (4202)	Travel-request lifecycle, budget calc; hosts the unified Home dashboard
reservation-management	8083	reservation-management-app (4203)	Reservation booking and PDF storage / download
reimbursement-management	8084	reimbursement-management-app (4204)	Expense-claim submission and processing

Mandatory startup order

account-management (creates and seeds the shared MySQL schema) -- auth-service -- travel-planner -- reservation-management -- reimbursement-management.

3. Technology Stack

Backend

Area	Technology / Version
Language and runtime	Java 21
Framework	Spring Boot 3.5.7, Spring Cloud 2025.0.0
Core modules	Spring Web (REST), Spring Security, Spring Data JPA
Inter-service calls	Spring Cloud OpenFeign (declarative HTTP clients)
Authentication	JWT 0.12.6 (HMAC-SHA256 signed JWT)
Database	MySQL 8 (mysql-connector-j, dialect MySQLDialect, ddl-auto=update)
API documentation	springdoc-openapi (Swagger UI)
Boilerplate reduction	Lombok 1.18.40
Build tool	Gradle (system Gradle; wrapper jar not committed)

Frontend

Area	Technology / Version
Framework	Angular 17.2 (NgModule-based)
Language	TypeScript 5.3
Reactive library	RxJS 7.8
UI / styling	Bootstrap 5 and Bootstrap Icons (CDN) + a custom vibrant theme
Charts	Chart.js 4 (CDN) - pie / doughnut, bar and line charts
HTTP / auth	HttpClient with auth + error interceptors; route + role guards; cross-app JWT hand-off
Tooling / test	Angular CLI 17, Karma + Jasmine

4. Backend Microservices in Detail

4.1 account-management (port 8081)

The master data store and security backbone. It owns the employee and grade tables and is the owner of the shared account_management MySQL database, which auth-service also uses.

- Entities: Employee, Grade (Grade-1/2/3), GradeHistory (refresh_tokens and token_blacklist live in this DB, owned by auth-service).
- Employee IDs use a JPA sequence starting at 100000 (always 6 digits). Emails must end with @cognizant.com; passwords are BCrypt-12 hashed.
- Endpoints: GET/POST/PUT/DELETE /api/employees, GET /api/employees/me, GET /api/employees/{id}, GET /api/grades, GET /api/gradeHistory.
- Seeds three grades and three default users on first startup.

4.2 auth-service (port 8080)

Centralised authentication. It issues and refreshes tokens and is the single source of truth for the logout blacklist. It connects directly to the shared account_management MySQL database and owns the refresh_tokens and token_blacklist tables.

- Issues a 1-hour access token and a 7-day refresh token on login; rotates the refresh token on use.
- Endpoints: POST /login, POST /auth/refresh, POST /auth/logout, GET /auth/blacklist/check.
- On logout, the refresh token is deleted and the access token is added to the blacklist.

4.3 travel-planner (port 8082)

Manages the travel-request lifecycle and budget calculation, and serves the unified Home dashboard for the front end. Fetches employee / grade data from account-management via Feign.

- Entities: TravelRequest, Location (8 seeded cities), TravelBudgetAllocation.
- Status flow: NEW to APPROVED / REJECTED. Priority ONE/TWO/THREE caps trip duration (30 / 20 / 10 days).
- Budget = grade daily cap (Grade-1 Rs. 15,000, Grade-2 Rs. 12,500, Grade-3 Rs. 10,000) times the number of days.
- Endpoints: POST /new, GET /{hrid}/pending, PUT /{trid}/update, POST /calculatebudget, GET /my, GET /approved, GET /{trid}, GET /locations.

4.4 reservation-management (port 8083)

Lets a TravelDeskExe upload booking confirmations (PDF) for approved trips and lets employees and the desk track them. Enforces budget caps per reservation type.

- Reservation types: Flight, Train, Bus, Cab, Hotel. PDF max 1 MB; stored on disk, served on demand.
- Reservations budget = 70% of the approved budget; caps: transport 35%, cab 15%, hotel 50%.
- Date rules: train/bus exactly 1 day before the from-date; hotel on the from-date. One reservation per category.
- Endpoints: POST /add, GET /track/{travelRequestId} (Employee + TravelDeskExe), GET /{id}, GET /{id}/download, GET /types.

4.5 reimbursement-management (port 8084)

Employees submit post-trip expense invoices with PDF proof; a TravelDeskExe approves or rejects each one. Enforces per-invoice ranges and daily cumulative limits.

- Types: Food, Water, Laundry, LocalTravel. PDF max 256 KB.
- Per-invoice: Food/Water Rs. 1,000-1,500; Laundry Rs. 250-500; LocalTravel up to Rs. 1,000. Daily caps: Food+Water 1,500/day; Laundry 500/day; LocalTravel 1,000/day.
- A reimbursement can only be raised once the travel request is APPROVED; only a TravelDeskExe can process it.
- Endpoints: POST /add, PUT /{id}/process, GET /my, GET /{travelRequestId}/requests, GET /{reimbursementId}, GET /types.

5. Front-end Applications

Five Angular 17 micro-frontends. After login every role lands on a unified, role-aware Home dashboard (travel-planner-app /home), and all apps share an identical data-driven navigation bar. Tokens are passed between apps as URL parameters during cross-app navigation and seeded into each app's localStorage before route guards run.

App (port)	Primary role	Key pages
auth-app (4200)	All	Login
account-management-app (4201)	HR	Employee list (+ role / grade charts), add employee, change grade
travel-planner-app (4202)	Employee / HR	Home dashboard, new request, my requests (+ status / priority charts), pending requests, calculate budget, request details
reservation-management-app (4203)	TravelDeskExe / Employee	Add reservation, view reservations (+ spend chart), my reservations (track), reservation details
reimbursement-management-app (4204)	Employee / TravelDeskExe	Submit claim, my claims (+ status / type / amount charts), process / search claims

Travel-request-ID fields on the Add Reservation, View Reservations, My Reservations and Process Claims pages are dropdowns (no manual ID typing), each populated from the relevant scoped endpoint.

6. Authentication and Authorization

- Stateless JWT signed with HMAC-SHA256 using a shared secret identical across all five services.
- Token claims: sub (email), role (HR / Employee / TravelDeskExe), iat, exp. Access token valid 1 hour; refresh token 7 days (rotated on use).
- Every protected service runs a JwtAuthFilter: it validates the signature and expiry locally and calls auth-service to check the blacklist on each request (fail-open if auth-service is unreachable).
- Spring Security maps the JWT role claim directly to an authority (no ROLE_ prefix). Outgoing Feign calls forward the bearer token via a FeignAuthInterceptor.
- The front end stores the token in localStorage, attaches it via an HTTP interceptor, guards routes by role, and hands the token between apps through URL params.

7. Database Design (MySQL 8)

The backend runs entirely on MySQL 8 over TCP 3306 (root user). Each service uses spring-boot-starter-data-jpa plus mysql-connector-j with ddl-auto=update, so Hibernate auto-creates and updates all tables on startup. There are four databases.

Service	MySQL database	Notes
account-management	account_management	Schema owner; shared with auth-service
auth-service	account_management (shared)	Creates refresh_tokens + token_blacklist in the shared DB
travel-planner	travel_planner	Own DB

Service	MySQL database	Notes
reservation-management	reservation_management	Own DB
reimbursement-management	reimbursement_management	Own DB

account-management and auth-service connect to the same account_management database directly over TCP 3306. A reference DDL script (BE/etd_mysql_schema.sql) reproduces the full schema (4 databases, 13 tables) for manual setup, but is optional because ddl-auto=update builds it automatically. Fresh start: DROP and re-CREATE the relevant database, then restart.

8. Inter-Service Communication (Feign)

Caller	Calls	Purpose
travel-planner	account-management, auth-service	Validate employee + grade; blacklist check
reservation-management	travel-planner, auth-service	Validate approved trip + budget; blacklist check
reimbursement-management	travel-planner, account-management, auth-service	Validate trip + dates; validate processor role; blacklist check

9. Key Business Rules

9.1 Grade changes (account-management)

- Grade can only be upgraded (a lower grade id = more senior), never downgraded.
- No grade change within 2 years of joining; thereafter at most once per year.
- A TravelDeskExe is always force-assigned to Grade-1 on creation.

9.2 Travel request and budget (travel-planner)

- Trip duration must not exceed the priority cap (ONE 30, TWO 20, THREE 10 days).
- Only the logged-in employee can raise their own request; the approver must have the HR role.
- Budget is computed only after approval, from the grade daily cap times the number of days.
- Hotel rating by role: HR may book 5-STAR / 7-STAR; others 3-STAR / 5-STAR.

9.3 Reservations and reimbursements

- Reservation budget caps (of 70% of approved budget): transport 35%, cab 15%, hotel 50%.
- A reservation or a reimbursement can only be made against an APPROVED travel request.
- Reimbursement per-invoice and daily-cumulative limits are enforced per expense type; remarks are required on rejection; only a TravelDeskExe can process a claim.

10. User Interface and Analytics

- A vibrant, modern theme shared across all five apps: animated gradient navbar, gradient buttons, glassmorphism cards, slide-in alerts, and a full-page animated travel-cityscape background image (SVG) behind every page.
- A unified, role-aware Home dashboard and an identical data-driven navigation bar across all apps.
- Chart.js analytics (pie / doughnut, bar and line) on data-rich pages: employees by role and grade; travel requests by status and priority; reimbursement claims by status, type and amount over time; reservation spend by type.
- Friendly emoji accents on titles, empty states and primary actions; fully responsive; respects reduced-motion preferences.

11. Default Seed Credentials (development)

Role	Email	Password	Employee ID
HR	admin.hr@cognizant.com	Admin@123	100000
TravelDeskExe	desk.exec@cognizant.com	Exec@123	100001
Employee	john.employee@cognizant.com	Employee@123	100002

12. Build, Run and Tooling

- Prerequisites: Java 21, system Gradle, MySQL 8 running on localhost:3306, and Node.js + Angular CLI 17 for the front end.

- Set the MySQL root password in each service's application.properties to match your instance; databases auto-create (createDatabaseIfNotExist=true).
- Backend: gradle bootRun per service, started in order (account-management first, then auth-service, then the rest).
- Frontend: npm install then npm start (ng serve) per app on ports 4200-4204.
- Swagger UI per service at /swagger-ui.html. Log in at http://localhost:4200; all roles land on the Home dashboard.

13. Current Stage, Limitations and Future Enhancements

- Development-stage configuration: the JWT secret and MySQL credentials are hard-coded in application.properties; externalise them for production.
- No central API gateway or service discovery: the browser calls each service directly over open CORS.
- Test coverage is limited to the default Spring context-load test in each service.
- Front-end shared code (navbar, config) is duplicated across the five apps (no shared library / Module Federation yet).
- Natural next steps: API gateway + service discovery, externalised secrets, integration / contract tests, a shared front-end library, and a dedicated aggregate analytics dashboard.

14. Resume Talking Points

Suggested project title

Employee Travel Desk (ETD) -- Microservices-Based Corporate Travel Management Platform

Tech line

Java 21, Spring Boot 3.5, Spring Cloud OpenFeign, Spring Security (JWT), Spring Data JPA, MySQL 8, Angular 17, TypeScript, Chart.js, REST, OpenAPI / Swagger, Gradle.

Achievement bullets (pick the ones that fit your role)

- Designed and built a 5-service microservices backend (Spring Boot 3.5, Java 21) on MySQL with declarative Feign inter-service communication.
- Implemented centralised, stateless JWT authentication (HMAC-SHA256, 1-hour access + 7-day refresh tokens) with token rotation and a server-side logout blacklist checked on every request.
- Enforced role-based access control for three roles (HR, Employee, TravelDeskExe) using Spring Security authorities derived from JWT claims.
- Developed 5 Angular 17 micro-frontends with a unified role-aware dashboard, consistent navigation, HTTP interceptors, route / role guards, and secure cross-app token hand-off.
- Modelled complex domain logic: grade-change governance, priority-based trip-duration caps, grade-driven budget calculation, and per-category reservation / reimbursement limits.
- Added Chart.js analytics (pie, bar, line) across the apps and standardised the backend persistence layer on MySQL 8 with Hibernate / JPA.

One-line version

Built a full-stack microservices travel-management system (5 Spring Boot services on MySQL + 5 Angular apps) with JWT-secured role-based access, Feign inter-service calls and Chart.js analytics.